Document number: P3264R1
Audience: EWG

Ville Voutilainen
2024-05-17

Double-evaluation of preconditions

Abstract

This paper explains why we might want to (continue to) allow precondition checks to be evaluated not once, but in some cases twice, and by that allowance, C++ users shouldn't try to write precondition checks where they rely on those checks being evaluated exactly once.

The implementation strategy that sometimes leads to such double-evaluation is one where precondition checks can be independently enabled in multiple different TUs, namely

- precondition checks can be performed in the calling-client, meaning that function call expressions are transformed into expressions that check a precondition first, and if the precondition isn't satisfied, call a violation handler, and then, call the function, if the precondition was satisfied or if the contract semantic is "observe" and the violation handler returned normally.
- precondition checks can be performed in the defining TU, meaning that all calls of the function have the precondition checked, because that check is generated as-if in the body of the function, and in practice, that as-if isn't just as-if, it's exactly that, the body of the function checks the precondition.
- both of these are independent, and there's no communication or orchestration between those separate TUs.
- so, if both of these independent checks are enabled, a precondition is checked twice, once at the call site, once in the definition.

This ends up being extremely useful in practical deployment scenarios, and moreover, ends up being extremely useful in such scenarios where the independent checks end up being enabled by a user who is not a programmer.

Use case 1: enable caller-client-side checks in an application without recompiling a library

So, in this use case, you have

- a library that has preconditions in its API function declarations
- an application that uses that library

Furthermore, you have a build of the library where checks are not enabled.

- Maybe your package manager doesn't install multiple versions of the same library (like they quite often don't).
- Maybe the library is installed on a system image that you can't modify.
- Maybe the library is just installed on a system you're investigating, and you just don't have a precondition-check-enabled library in hand.

The application misbehaves, and you want to investigate why. You can either compile the application, or your friends back at the R&D office can, or they have given you both a checking and non-checking version of the application. But you can't recompile the library, and can't install a different version of it. You don't own it, you don't have the source code, you don't have a check-enabled version of it.

All you need to do is replace the application with one that has precondition checks for function calls enabled in the calling TU, and run it. Collect violation information when contract violations occur, and perform the subsequent steps of your investigation based on that.

Use case 2: enable definition-side checks in a library without recompiling an application

So, in this use case, you have

- a library that has preconditions in its API function declarations
- an application that uses that library

Furthermore, you have a build of the application where checks are not enabled.

- Maybe your package manager doesn't install multiple versions of the same application (like they quite often don't).
- Maybe the application isn't yours, and you just don't have a precondition-check-enabled application in hand.

The application misbehaves, and you want to investigate why. You can either compile the library, or your friends back at the R&D office can, or they have given you both a checking and non-checking version of the library. But you can't recompile the application, and can't install a different version of it. You don't own it, you don't have the source code, you don't have a check-enabled version of it.

All you need to do is replace the library with one that has precondition checks for function definitions enabled in the defining TU, and re-run the application. Collect violation information when contract violations occur, and perform the subsequent steps of your investigation based on that.

The potentially surprising Use case 3: enable both caller-client-side checks and definition-side checks

Neither of those previous use cases lead to double-evaluation as such. But what if you have checks enabled on the one side of the TU equation, and you decide to enable them on the other side as well?

Again, maybe you can't control the one side of the TU equation, and can't change it. But you want more information, so you enable the checks on the other side of the TU equation too. You might want to do this to get more information, such as

- more precise source location of the call
- or more precise source location of the function called
- precondition violation information of an indirect call that isn't otherwise checked at the calling-client-side

You can just enable the check in the binary you control, and re-run the application+library combination. But now it will evaluate some precondition checks twice.

Virtual functions

We have a couple of proposals that propose that virtual functions should have two kinds of checks:

- ones that are checked for the entry point of the call, for the statically chosen function
- ones that are checked for the overrider dispatched to, i.e. for the dynamically chosen function.

If the call and the definition of the overrider are in separate TUs, but the statically chosen function and the overrider end up being the same, you end up checking the same precondition twice. And the call site might not be able to see that the statically chosen function and the overrider are the same. The definition side has no idea.

But there are implementation approaches that avoid this, and *guarantee* single evaluation..

Are there? Do they really allow *both* use case 1 and use case 2? Even if they do, at what cost?

I have seen various variations of such a guarantee being suggested.

- One of them suggests that the calling client should call either the plain function, or an inline wrapper that performs a precondition check.
  - That sure sounds to me like the approach doesn't support enabling/disabling checks on the library side, without recompiling the application.
- Another suggests that a library should expose either the plain function or a wrapping function that performs the check, and thus the definition-side of the TU equation can turn checks on and off.
  - That sure sounds to me like the approach doesn't support enabling/disabling checks on the application side, without recompiling the library.
- I have seen at least two additional alternative suggestions, where the need to check is checked by a program-unique function, i.e. it's a run-time check whether to perform checks, and with some orchestration, multiple separate TUs can be made to figure out whether they need to perform the check or not, or that there would be some TLS data that tells whether to perform the check, as an alternative suggestion that has less ABI impact and less symbol impact.

Alright then. Let's talk about the costs of those things.

The cost of an evaluate-exactly-once guarantee

For the static approaches where exactly one side controls whether checks are on or off, the cost is that you need to recompile that one side when you need to flip precondition checks on or off. In some deployment scenarios, that cost is an insurmountable mountain you can't climb, because you might not be able to compile that side of the TU equation, because you don't own it. Or even if you sometimes can, you may be in a situation where you need to get investigation results ASAP, and don't have time or the right machines or the access to them to recompile.

For any dynamic approach, consider this example, categorically:

```
void f(int x) pre(x >= 0);
```

The check is simple. It's not performing a huge computation, it's not doing complicated things. Do you really want to pay a cost of a run-time operation that guarantees exactly-once-evaluation for it? Wouldn't you just want to inline that check to wherever it's performed? Would you expect that many of your preconditions are like that? Wouldn't you want them to be as low-overhead as possible?

In contrast, evaluating such simple conditions twice or more than twice has probably negligible additional costs; the values are in the cache, your branch predictor is warm.

The advantages of allowing double-evaluation, recap

The advantages of an implementation approach where precondition checks are independently possible per-TU, without any attempt to coordinate them, are thus:

- Deployment flexibility; you can enable checks in a caller-client even if your library doesn't have them enabled, without recompiling the library. Likewise, you can enable checks in a defining-library even if your application doesn't have them enabled, without recompiling or relinking the application.
- If you want, you can do both. You can enable checks on the caller-client side, or the defining-library side.
- None of these enablings/disablings of checking in one side or both sides of a TU equation causes an ABI break.
- Some of these examples are simple application-library pairs, but the advantages apply equally well to graphs of libraries.
- When checks are enabled, they are efficient. There are no mandated run-time calls to any guarantee-single-eval facilities, there is no TLS overhead.

I quite plainly expect these advantages to be so compelling that even if the standard mandates a requirement of exactly-once evaluation, such an implementation approach will be provided anyway. As a non-conforming extension, if need be.

What would the standard actually guarantee?

If my guesstimate of how compelling the aforementioned advantages are ends up hitting the mark, what value is there for the standard to guarantee exactly-once evaluation?

It would guarantee exactly-once evaluation for all C++ code built with a conforming implementation. But if the non-conforming approach is as compelling as expected, then in the ecosystem, the guarantee won't hold. There are going to be builds and deployment scenarios where it doesn't hold. So the guarantee would hold on paper, in theory, but not in practice, not in the wild.

Practical packaging ruminations

As mentioned, it's often the case that Linux package systems do not package multiple different builds of the same application or a library. There are exceptions to that, but by and large they don't.

So, it wouldn't be entirely unfathomable to have the packages be built with precondition checks disabled, and the vendor telling you that you need to enable caller-client-side preconditions to get checks.

But that's not the whole packaging story. Fedora/RHEL have been enabling various kinds of run-time checks for quite some time. Quoth a vendor:

```
we build the whole distro with similar flags to what GCC 14's -fhardened does
-D_FORTIFY_SOURCE=3 -D_GLIBCXX_ASSERTIONS -fstack-clash-protection -fcf-protection -Werror=format-security and more
and then scan all the binaries to make sure every object compiled from C, C++ or asm used those flags
```

So, it's equally fathomable that some vendors might build their libraries with precondition checks enabled. And some others might build theirs with precondition checks disabled.

An unorthodox twist

There's an additional packaging rumination. A vendor might indeed package libraries with checks enabled, but without requiring in any way that the applications that use such libraries are C++26 applications. They might be C++20, they might be C++11, they might be C++03.

This would mean that for those applications, the declarations of the functions defined in the library don't have contract assertions, because they are naturally ill-formed in pre-C++26 programs. But the library could be built with declarations that have contract assertions.

Yes, I know what various committee members will say, they have already said it. "That's an ODR violation, that violates the spec, declarations must agree, that's IFNDR, you Can't Do That!"

And yet, with the implementation approach where TUs can enable checks independently from other TUs, all that works fine. The implementation doesn't diagnose the IFNDR, and its definition of the resulting UB is to just run the code. And it'll work perfectly fine. Your C++20 and earlier applications can trigger precondition violations and get the benefits of those run-time checks even though they are completely unaware of what C++26 is and what contracts are.

What can you actually achieve with an evaluate-exactly-once guarantee?

My spoiler alert for this question is "not much". Every suggested use case for being able to rely on evaluate-exactly-once seems to be about establishing some sort of state in a precondition, and unraveling that state in a postcondition.

Well. Now we have two major problems:

1. This doesn't work if your function throws exceptions. When an exception is thrown, a postcondition check isn't performed. Any attempt to reset the state established by a precondition in a postcondition will fail to work correctly.
2. Look at the use cases explained in this document again. You'll notice that they talk about enabling *precondition* checks. In none of those descriptions are postcondition checks mentioned at all. And there's a reason for that. It's a very plausible deployment scenario that

> precondition checks are on and postcondition checks are off. Because precondition checks tell an application developer/bughunter whether the application is calling a library correctly, or from the other perspective, they tell a library developer/bughunter whether an application is calling the library correctly. But postconditions don't tell any such thing, they tell you whether a library has a bug. It seems far more often likely that precondition checks are on than is the case for postcondition checks. And then, if you try to do that stateful programming with precondition/postcondition pairs, you'll just fail to do it correctly.

Summa summarum

An implementation approach where double-evaluation of preconditions sometimes, but not always happens has multiple compelling advantages and benefits.

In contrast, in the presence of such an implementation approach, and the chance of it having been deployed in the wild, it would be rather unwise to rely on a precondition check being evaluated exactly once.

To me, that's quite an acceptable trade-off. It is, for various reasons, very unwise in general to have precondition checks that break your program if they are evaluated twice in a row. Such preconditions are likely going to hurt your ability to reason about those checks in some cases, and they are likely going to hurt the ability of tools to reason about them too.

It's more unwise still to rely on a postcondition check undoing state transitions or resource acquisitions or anything like that performed by a precondition. An exception thrown will stop such techniques from working, and plausible deployment scenarios where preconditions are checked but postconditions are not checked will also stop such techniques from working.

The advantages of possible double-evaluation of preconditions outweigh the disadvantages.

Oh, and you were all just dying to ask, all this time: does the same apply to contract_asserts and postconditions? As far as I can see.. ..no. :) At least not to the same extent. But it's certainly arguably so that similar advantages can be achieved for postconditions as well.